

EDS Lab Assignments

Practical 2

Name: Sakshi Shivaji Mulay

PRN: 202501040061

2.1.1 List Operations

Problem Statement:

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu.

Depending on the choice selected, the program should perform the following actions:

- Add: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- Remove: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- Display: Displays the current list of integers. If the list is empty, display "List is empty".
- Quit: Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Code: l1 =

[] while

True:

 print("1. Add")

print("2. Remove")

print("3. Display")

 print("4. Quit")

 n=int(input("Enter choice: "))

if n==1:

 add = int(input("Integer: "))

 l1.append(add)

```

        print(f"List after adding: {l1}")
elif n==2:
    if len(l1)==0:
        print("List is empty")
    elif len(l1)!=0:
        remove=int(input("Integer: "))
        if remove not in l1:
            print("Element not found")
        else:
            l1.remove(remove)
            print(f"List after removing: {l1}")
elif n==3:
    if len(l1)==0:
        print("List is
empty")
    elif n==4:
        print(l1)
        break
    else:
        print("Invalid choice")

```

Output:

The screenshot displays a coding platform interface with a dark theme. On the left, a sidebar shows the problem statement for '2.1.1. List operations'. The main area is divided into three sections: a code editor, a test results panel, and a terminal.

Problem Statement: Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- Quit:** Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they

Code Editor: The code editor shows the following Python code:

```

1 # write the code..
2 l1=[]
3 while True:
4     print("1. Add")
5     print("2. Remove")

```

Test Results: The test results panel shows that the code has passed 2 out of 2 shown test cases and 1 out of 1 hidden test cases. The average time is 0.202 s and the maximum time is 0.335 s.

Test Case 1: The test case shows the expected output and actual output for the first test case. The expected output is:

```

1. Add
2. Remove
3. Display
4. Quit
Enter-choice: 1
Integer: 11

```

The actual output is:

```

1. Add
2. Remove
3. Display
4. Quit
Enter-choice: 1
Integer: 11

```

Terminal: The terminal shows the output of the program, which matches the expected output for the first test case.

2.1.2 Dictionary Operations

Problem Statement:

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs. The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Code:

Initial dictionary with 10 predefined records

```
student = { 1: "Amit",
            2: "Riya",
            3: "Kiran",
            4: "Neha",
            5: "Arjun",
            6: "Pooja",
            7: "Rahul",
            8: "Sneha",
            9: "Vikram",
            10: "Anjali"
}
```

write your code here..

```
print("Original Dictionary:", student)
```

```

new_key    =    int(input())
new_value  =    input()
student[new_key] = new_value
print("After Insertion:", student)

update_key = int(input())
new_val = input() if
update_key in student:
    student[update_key] = new_val
print("After Update:", student)

del_key = int(input())
if del_key in student:
    del student[del_key]
print("After Deletion:", student)

print("Traversing Dictionary:")
for key, value in student.items():
    print(f"{key} : {value}")

```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for "2.1.2. Dictionary Operations" is visible, detailing the requirements for insertion, update, deletion, and traversal. The main editor shows a Python script that implements these operations. The script takes an initial dictionary with 10 predefined records and performs the requested actions based on user input. The right sidebar shows the test results, indicating that 2 out of 2 shown test cases and 2 out of 2 hidden test cases passed. The test case details show the expected and actual output, which match perfectly.

2.1.2. Dictionary Operations 45:35

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after

DictOper... Submit

```

18 key1 = int(input())
19 value1 = input()
20 student.update({key1:value1})
21 print("After Insertion:",student)
22

```

Average time 0.053 s 53.25 ms Maximum time 0.059 s 59.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 49 ms Debug

Expected output

Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Anjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}

Actual output

Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Anjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}

11 Suresh

After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Anjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

Terminal Test cases

< Prev Reset Submit Next >

2.2.1 Linear Search Technique

Problem Statement:

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index.
- If the element is not found, print Not found.

```
Code: def linear_search(arr,
key):
    for i in
range(len(arr)):
    if arr[i] == key:
        return i
    return -1
```

```
arr = list(map(int,input().split()))
key = int(input())
```

```
result = linear_search(arr, key)
if result != -1:
    print(result)
else:
    print("Not found")
```

Output:

The screenshot displays the CodeTANTRA online IDE interface. On the left, the problem statement for '2.2.1. Linear search Technique' is shown, including the input/output format and a sample test case. The main editor shows the Python code for the linear search algorithm. The right sidebar displays the execution results, indicating that 2 out of 2 shown test cases passed and 2 out of 2 hidden test cases passed. The test cases show the input array [1, 2, 3, 4, 3, 5, 6] and the key 3, with the expected output being the index 3.

CTP1709...

```
1 def linear_search(arr, key):
2     for i in range(len(arr)):
3         if arr[i] == key:
4             return i
5     return -1
```

Average time: 0.025 s (25.50 ms) | Maximum time: 0.030 s (30.00 ms)

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 (24 ms) [Debug]

Expected output	Actual output
1 2 3 4 3 5 6	1 2 3 4 3 5 6
3	3
2	2

Test case 2 (30 ms)

Terminal | Test cases

< Prev | Reset | Submit | Next >

2.2.2 Captain of the Team

Problem Statement:

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

Code:

```
heights = list(map(int,input().split())) tallest_player
= max(heights) print(tallest_player)
```

Output:

The screenshot shows the CodeTANTRA online coding environment. On the left, the problem description for '2.2.2. Captain of the Team' is displayed, including the input and output formats. The main editor on the right contains the Python code:

```
1 height=list(map(int,input().split()))
2 tallest_player = max(height)
3 print(tallest_player)
```

 Below the code editor, the execution results are shown: 'Average time 0.012 s', 'Maximum time 0.012 s', and '1 out of 1 shown test case(s) passed'. A detailed view of 'Test case 1' shows the 'Expected output' as '171 169 185 156 174 191 186 190 187 172 160' and the 'Actual output' as '171 169 185 156 174 191 186 190 187 172 160', with the value '191' highlighted in both. The interface includes a top navigation bar with 'Home', 'Learn Anywhere', and a user profile, and a bottom bar with 'Prev', 'Reset', 'Submit', and 'Next' buttons.